

1. DEPENDANCES ENTRE INSTRUCTIONS

En cours, le cas de dépendances 2 instructions arithmétiques a été étudié. Le but de cet exercice est de généraliser cette étude à l'ensemble des 4 catégories d'instructions (Arithmétique, Load, Store, Branch).

- 1) Dans le cas où la première instruction est de type Load, analyser les différents types de dépendances possibles suivant le type de la deuxième instruction (Arithmétique, Load, Store, Branch). Etudier l'impact sur le comportement des instructions dans le pipeline.
- 2) Dans le cas où la première instruction est de type Store, analyser les différents types de dépendances possibles suivant le type de la deuxième instruction (Arithmétique, Load, Store, Branch). Etudier l'impact sur le comportement des instructions dans le pipeline.

1) ① Load puis arithmétique

LOAD	IF	ID	EX	MEM	WB		
Arithmétique		IF	ID	STALL	EX	MEM	WB

$I_1: (A) \rightarrow R_4$ (LOAD)

$I_2: R_4 + R_5 \rightarrow R_6$ (ARITHM)

} RAW R_n dans registre (A) mémoire

LOAD	IF	ID	EX	MEM	WB		
Arithmétique		IF	ID	EX	MEM	WB	

$I_1: (A) \rightarrow R_4$

$I_2: R_3 + R_5 \rightarrow R_4$

} WAW

② Load puis load

(Si I_2 utilise le résultat de I_1 (un adresse), c'est RAW)

$I_1: (A) \rightarrow R_4$

$I_2: (B) \rightarrow R_4$

} WAW

③ Load puis store

LOAD	IF	ID	EX	MEM	WB		
STORE		IF	ID	STALL	EX	MEM	WB

$I_1: (A) \rightarrow R_1$

$I_2: R_1 \rightarrow (B)$

} RAW

④ Load puis branche

LOAD	IF	ID	EX	MEM	WB		
Branch		IF	ID	STALL	EX	MEM	WB

$I_1: (A) \rightarrow R_1$

$I_2: R_1 \text{ compare LABEL}$

} RAW

Aléa de contrôle : si la prédiction est fausse, flush des instructions spéculatives.

2) ① Store puis arithmétique

$I_1: R_1 \rightarrow (A)$

$I_2: R_2 + R_3 \rightarrow R_1$

} WAR

schéma sans stall

2. Store puis Arithmétique 存储器，然后算术

(2.1) Cas “WAR” sur un registre lu par le store (dépendance de nom)

(2.1) 存储器读取寄存器时出现“WAR”情况（名称依赖性）

Ex :例：

- I1 : ST (A) ← R1
- I2 : R1 ← R2 + R3

Dépendance : **WAR** (I1 lit R1, I2 écrit R1). 依赖关系：WAR (I1 读取 R1, I2 写入 R1)。

Impact pipeline : en **pipeline in-order**, souvent **pas de stall** (I1 lit R1 en ID avant que I2 n’écrive en WB). 影响管道：在顺序管道中，通常不会出现停滞（I1 在 ID 中读取 R1 之前，I2 不会在 WB 中写入）。

Schéma (sans stall, in-order) : 流程图（无停滞，顺序执行）：

- STORE : IF ID EX MEM WB 商店：IF ID EX MEM WB
- ARITH : IF ID EX MEM WB

(Option phrase qui sécurise : “En OoO, renommage supprime WAR/WAW”.)

(安全提示选项：“在 OoO 中，重命名会删除 WAR/WAW”。)

2. Store puis Load 存储后加载

(2.3) Cas RAW mémoire (même adresse) (2.3) 内存 RAW 情况（相同地址）

- I1 : ST (A) ← R1
- I2 : R2 ← (A)

Dépendance : **RAW mémoire** (le load doit lire la valeur stockée).

依赖性：RAW 内存（加载操作必须读取存储的值）。

Impact pipeline : le load peut devoir **attendre** la disponibilité du store ; avec **store-to-load forwarding** (store buffer → load), ça peut éviter (ou réduire) le stall.

影响管道：加载操作可能需要等待存储器的可用性；通过存储器到加载的转发机制（存储器缓冲区→加载），可以避免（或减少）停滞现象。

Schéma “safe” (avec 1 STALL possible si adresse/donnée pas prêtes assez tôt) :

安全模式（若地址/数据未及时准备就绪，可能出现 1 次停顿）：

- STORE : IF ID EX MEM WB 商店：IF ID EX MEM WB
- LOAD : IF ID STALL EX MEM WB

Note : si le cours dit “store-to-load forwarding” + adresse connue tôt, tu peux commenter : “STALL

peut être évité”. 注：若课程提及“店到货转运”+提前知晓地址，可补充说明：“可避免 STALL”。

2. Store puis Store 商店，然后商店

(2.5) Cas WAW mémoire (même adresse) (2.5) WAW 内存情况（相同地址）

- I1 : ST (A) ← R1
- I2 : ST (A) ← R2

Dépendance : **WAW mémoire** (deux écritures sur la même case ; l’ordre programme doit être respecté, la 2e “gagne”).

依赖性：WAW 内存（同一单元格的两次写入；必须遵循程序顺序，第二次写入“胜出”）。

Impact pipeline : en in-order, pas besoin de stall conceptuel ; en pratique, les stores passent par un buffer et “commit” en ordre.

影响管道：在顺序模式下，无需概念性停滞；实际操作中，存储器通过缓冲区并按顺序提交。

Schéma (souvent sans stall) : 模式（通常无失速）：

- STORE1 : IF ID EX MEM WB
- STORE2 : IF ID EX MEM WB

2. Store puis Branch 商店然后分店

(2.7) Cas “indépendant” (le branch ne teste pas un registre touché)

(2.7) “独立”情况（分支不测试受影响的寄存器）

- I1 : ST (A) ← R1
- I2 : BEQ R2, R3, LABEL

Dépendance : aucune (entre store et branch). 依赖性：无（在存储器和分支之间）。

Impact pipeline : seulement **aléa de contrôle** (prédiction/flush), mais pas causé par le store.

影响管道：仅为控制随机性（预测/刷新），但并非由存储器引起。

Schéma (sans stall, juste rappeler flush possible) : 示意图（无失速，仅需注意可能的冲洗）：

- STORE : IF ID EX MEM WB 商店：IF ID EX MEM WB
- BRANCH : IF ID EX MEM WB 分支：IF ID EX MEM WB

Phrase : “Aléa de contrôle : si prédiction fausse → flush des instructions spéculatives.”

控制随机性：若预测错误→清空投机性指令。

2. REORGANISATION DE LA STRUCTURE DU PIPELINE

Le but de cet exercice est de réorganiser la structure du pipeline plus précisément de permuter les étages EX et MEM. La nouvelle structure du pipeline serait : IF, ID, MEM, EX, WB. Nous supposons que le calcul d’adresse est effectué durant la phase ID.

- Dans le cas où la première instruction est de type Load, et suivant le type de la deuxième instruction (Arithmétique, Load, Store), étudier l’impact de la nouvelle structure de pipeline sur l’exécution des instructions.
- Dans le cas où la première instruction est de type Store, et suivant le type de la deuxième instruction (Arithmétique, Load, Store), étudier l’impact de la nouvelle structure de pipeline sur l’exécution des instructions.
- Dans le cas où la première instruction est de type Arithmétique, et suivant le type de la deuxième instruction (Arithmétique, Load, Store), étudier l’impact de la nouvelle structure de pipeline sur l’exécution des instructions.
- Comparer la nouvelle et l’ancienne structure de pipeline.

(1.a) Load → Arithmétique (RAW registre) (1.a) 加载 → 算术运算 (RAW 寄存器)

Ex : LD r1, [A] puis ADD r2, r1, r3. 例如： LD r1, [A] 然后 ADD r2, r1, r3 。

- Dépendance : **RAW** sur r1. 从属关系：RAW 在 r1 上。
- Impact (nouveau pipeline) : souvent **plus de stall** (ou moins qu'avant), parce que r1 est lu en MEM (étage 3) et l'ADD consomme en EX (étage 4), donc il y a naturellement **1 cycle d'écart**.

影响（新流水线）：通常会增加停滞（或比之前减少），因为 r1 在 MEM（第三级）读取，而 ADD 在 EX（第四级）消耗，因此自然存在 1 个周期的延迟。

Schéma typique (sans stall) : 典型示意图（无失速）：

- I1 LD : 1 IF | 2 ID | 3 MEM | 4 EX | 5 WB I1 LD : 1 输入 | 2 识别 | 3 存储 | 4 扩展 | 5 白平衡
- I2 ADD: 2 IF | 3 ID | 4 MEM | 5 EX | 6 WB I2 添加: 2 接口 | 3 标识 | 4 内存 | 5 扩展 | 6 白板

La donnée chargée à la fin de MEM (cycle 3) peut être forwardée pour l'EX du cycle 5 (il y a même “de la marge”).

在 MEM 结束时（第 3 周期）加载的数据可转发至第 5 周期的 EX（甚至还有“余量”）。

Conclusion : **amélioration** vs ancien pipeline (où load-use imposait souvent 1 bulle).

结论：相较于旧管道（其中负载使用常强制要求 1 个气泡）的改进。

(1.b) Load → Load (1.b) 加载 → 加载

Ex : LD r1, [A] puis LD r2, [B] 例如： LD r1, [A] 然后 LD r2, [B]

- Si registres dest différents et adresses indépendantes : **pas de dépendance** → pas de stall.
若不同寄存器和独立地址：无依赖关系 → 无停滞。
- Si I2 utilise r1 pour son adresse (ex LD r2, [r1]) : **RAW (adresse)** → I2 ne peut pas terminer son ID (calcul adresse) avant r1 dispo. Dans ce nouveau modèle, r1 n'est prêt qu'après MEM de I1, donc **stall(s) possibles en ID de I2**.
若 I2 使用 r1 作为其地址（例如 LD r2, [r1]）：RAW（地址）→ I2 无法在 r1 可用前完成 ID（地址计算）。在此新模型中，r1 仅在 I1 的 MEM 完成后才就绪，因此 I2 的 ID 可能出现延迟。
- Si même registre dest (r1 puis r1) : **WAW** (dépendance de nom), généralement **pas de stall en in-order**. 若相同寄存器 dest (r1 然后 r1)：WAW（名称依赖），通常在顺序执行中不会发生停滞。

(1.c) Load → Store (1.c) 加载 → 存储

Ex : LD r1, [A] puis ST [B], r1 例如： LD r1, [A] 然后 ST [B], r1

- Dépendance : **RAW** (valeur du store) si le store écrit r1 ; éventuellement RAW (adresse) si [r1].
依赖项：RAW（存储器值）如果存储器写入 r1；可能为 RAW（地址）如果 [r1]。
- Impact : en général **pas de longue série de stalls**. Le store a besoin de la donnée au moment où il “alimente” l'accès mémoire (selon modèle : lecture en ID ou forwarding vers MEM). Avec cette structure, la valeur du load arrive tôt (MEM), donc **0 ou 1 bulle max** selon les hypothèses de forwarding “store-data”.

影响：通常不会出现长时间的停滞。存储器在“为”内存访问提供数据时需要该数据（根据模型：ID 读取或转发至 MEM）。采用这种结构时，加载值会较早到达（MEM），因此根据“存储数据”转发假设，最多出现 0 或 1 个气泡。

(2.a) Store → Arithmétique (2.a) 存储器 → 算术

Ex : ST [A], r1 puis ADD r1, r2, r3 例如： ST [A], r1 然后 ADD r1, r2, r3

- Dépendance : **WAR** sur r1 (I1 lit r1, I2 écrit r1) — dépendance de nom.
依赖关系：WAR 对 r1（I1 读取 r1, I2 写入 r1）——名称依赖。
- Impact : en **in-order**, souvent **aucun stall** (I1 lit r1 en ID avant que I2 n'écrive plus tard en WB).
影响：在顺序模式下，通常不会出现停滞（I1 在 ID 中读取 r1，而 I2 稍后在 WB 中写入）。

(2.b) Store → Load (2.b) 存储 → 加载

Ex : ST [A], r1 puis LD r2, [A] 例如： ST [A], r1 然后 LD r2, [A]

- Dépendance : **RAW mémoire** (même adresse). 依赖性：RAW 内存（相同地址）。
- Impact : le load doit voir la valeur du store → il peut **attendre** si le store n'a pas encore “commité” sa donnée ; avec *store-to-load forwarding* (store buffer → load), le stall peut être **réduit/évité**.
影响：加载操作必须看到存储器的值 → 若存储器尚未“提交”数据，加载操作可进入等待状态；通过存储器到加载的转发机制（存储器缓冲区 → 加载），可减少/避免等待时间。
- Si adresses différentes (ou prouvées non-aliasées) : pas de dépendance mémoire.
若地址不同（或已证实非别名）：无内存依赖。

(2.c) Store → Store (2.c) 商店 → 商店

Ex : ST [A], r1 puis ST [A], r2 例如： ST [A], r1 然后 ST [A], r2

- Dépendance : **WAW mémoire** (même adresse). 依赖性：WAW 内存（相同地址）。
- Impact : en in-order, l'ordre programme est respecté → pas de stall “conceptuel” ; en pratique, commit en ordre via buffer.
影响：在顺序执行模式下，程序顺序得到遵守 → 不存在“概念性”停滞；实际操作中，通过缓冲区按顺序提交。

(3.a) Arithmétique → Arithmétique (RAW) (3.a) 算术 → 算术 (RAW)

Ex : ADD r1, r2, r3 puis SUB r4, r1, r5 例如： ADD r1, r2, r3 然后 SUB r4, r1, r5

- Dépendance : **RAW**. 附属建筑：RAW。
- Impact : généralement **pas de stall** (résultat prêt en EX de I1, et I2 consomme en EX un cycle plus tard). Ici EX est plus tard (étage 4), mais l'écart reste d'1 cycle entre les EX successifs : ça passe avec forwarding.
影响：通常不会出现停滞（I1 的 EX 结果已准备就绪，而 I2 在 EX 中消耗一个周期后）。此处 EX 发生较晚（第 4 级），但相邻 EX 之间仍保持 1 个周期的间隔：通过转发机制可实现。

(3.b) Arithmétique → Load (3.b) 算术 → 加载

Ex : ADD r1, r2, r3 puis LD r4, [r1] 例如： ADD r1, r2, r3 然后 LD r4, [r1]

- Dépendance : **RAW (adresse)** : le load a besoin de r1 en ID (car adresse calculée en ID).
依赖项：RAW（地址）：加载需要 ID 为 r1（因为地址是根据 ID 计算的）。
- Impact : c'est un point “piégeux” du nouveau pipeline : I2 doit avoir r1 **dès ID**, alors que I1 ne produit r1 qu'en EX (cycle plus tard). Donc **stall en ID de I2** est probable (au moins 1 bulle) si on ne peut pas forwarder vers ID.
影响：这是新管道中的一个“陷阱点”：I2 必须在 ID 阶段就获得 r1，而 I1 仅在 EX 阶段（稍后的周期）才产生 r1。因此，如果无法向 ID 阶段转发，I2 在 ID 阶段很可能出现停滞（至少 1 个气泡）。

(3.c) Arithmétique → Store (3.c) 算术 → 存储

Ex : ADD r1, r2, r3 puis ST [B], r1 例如： ADD r1, r2, r3 然后 ST [B], r1

- Dépendance : **RAW (valeur du store)**. 依赖项：RAW（存储器值）。
- Impact : souvent OK sans stall si la valeur peut être forwardée jusqu'au moment où le store en a besoin (selon modèle : lecture en ID vs forwarding vers MEM). Si le store exige la valeur en ID strict, alors **bulle(s) en ID** peuvent apparaître.
影响：通常在不发生停滞的情况下可行，前提是值可被转发至存储器需要时（取决于模型：ID 读取 vs 转发至内存）。若存储器要求严格的 ID 值，则可能出现 ID 气泡。

4. Comparer ancienne vs nouvelle structure 比较旧结构与新结构

- Gros avantage : **Load → Arith** (load-use) est **nettement moins pénalisant** (souvent 0 stall au lieu de 1). 重大优势：Load → Arith（加载-使用）的惩罚明显较小（通常为 0 次停滞而非 1 次）。
- Gros inconvénient : tout ce qui dépend d'un registre **pour calculer une adresse en ID** (ex ADD r1, ... puis LD [r1]) devient plus fragile : on veut une valeur “trop tôt” (en ID), donc **stall(s) en ID** plus fréquents si pas de forwarding vers ID.
重大缺点：所有依赖于寄存器计算 ID 地址的操作（例如 ADD r1, ... 然后 LD [r1]）都变得更脆弱：当需要“过早”获取 ID 值时，若未转发至 ID，则 ID 中的 stall(s) 会更频繁地发生。
- Autre effet : l'ALU (EX) arrive plus tard, donc certaines décisions “logiques” (comparaisons, branches si on les résout en EX) peuvent être **décalées**, avec impact possible sur le contrôle (selon le cours).

另一种影响：ALU（EX）处理稍晚，因此某些“逻辑”决策（如比较操作、分支判断若在 EX 中解决）可能被延迟，这可能会对控制产生影响（具体取决于课程内容）。